



introduzione

in questo corso vedremo:

- cosa deve fare una rete
- non vedremo specifiche implementazioni (sono troppi)

9 crediti, in un mondo ideale terminiamo prima di Natale

ricevimento lunedì 9-11, preferito scrivere via mail prima

ci saranno delle esercitazioni (dal dr. Matteo Paier)

si comunica principalmente su Teams, stesso posto mette le slides (mette quelle del anno precedente e le aggiorna man mano che facciamo le lezioni)

consiglia il libro "[Reti di calcolatori](#)" di [Peterson e Davies](#)



Esame

18 domande con esercizi/teoria

- orale a descrizione del prof
- voto tra 17 e 19 orale obbligatorio
- con voto < 17 non si è ammessi al orale
- **con voto < 10 non puoi iscriverti all'esame successivo (Anche sessione dopo)**

senza aver passato sistemi operativi non si può fare questo esame



Cosa è una rete

Formata da nodi e collegamenti, i nodi hanno capacità computazionale i collegamenti hanno capacità di trasporto.

argomentabilmente le prime effettive reti come le intendiamo noi risalgono nei primi 800, con i centralinisti e le linee telefoniche, il concetto di rete è molto flessibile e si può applicare a contesti anche non inerenti all'informatica.

reti inserire nome sono reti che gestiscono componenti meccaniche

in una rete bisogna assumere che ci saranno dei guasti/errori, quindi va progettata tenendo conto di questo, idealmente rendendola capace di risolvere i guasti automaticamente, senza intervento umano.

le reti di calcolatori sono generiche, quindi non specializzate per uno specifico scopo e devono essere flessibili (adattarsi e modificarle dinamicamente in base a nuove esigenze).



Performance



Latenza

Latency = Propagation Time + Transmit Time + Queue Time

- **propagazione**: tempo che impiega un bit a percorrere il collegamento
- **trasmissione**: tempo che impiega un bit a essere trasmesso dal nodo
- **tempo di accodamento**: tempo che impiega un bit a essere accettato dal nodo

se noi inviamo un pacchetto di piccole dimensioni, la latenza sarà principalmente determinata dalla propagazione (un principio fisico), il limite massimo della velocità di propagazione è la velocità della luce nel **vuoto** (ca. 300.000 km/s).

la luce nelle **fibre ottiche** però è inferiore a quella del vuoto

Se usassimo altri metodi di trasmissione (tipo usando rame) la velocità massima teorica diminuirebbe ulteriormente.

nota: un buon cavo di rame ha una velocità pari al 79% della velocità della luce nel vuoto, mentre una fibra ottica ha una velocità del 67% della velocità della luce nel vuoto.
ma perchè usare la fibra ottica allora?

la fibra ottica ha una **capacità di trasporto** molto maggiore del rame

quindi se voglio inviare pochi bit conviene il rame, se invio pacchetti grossi invece la fibra ottica è più efficiente.

è importante trovare il giusto bilanciamento tra latenza e capacità di trasporto, senza cercare di ottimizzare un aspetto quando un altro lo sovrasta.

Jitter:

quanto varia la latenza nel tempo, per alcuni contesti non è eccessivamente importante (scaricare o inviare files), mentre per altri contesti è fondamentale avere un valore stabile (telefonate, videochiamate, streaming).

avviene perchè internet è parecchio caotico, quindi i pacchetti non è detto che arrivino nello stesso ordine in cui sono stati inviati o che abbiano lo stesso intervallo di tempo con cui sono stati inviati.

ritardo X lunghezza di banda:

come una tubatura, una connessione ha una certa capacità di trasporto (larghezza della tubatura) e un certo ritardo (lunghezza della tubatura).

allo stesso modo una tubatura ha una certa quantità di acqua al suo interno, una connessione ha una certa quantità di dati al suo interno.

$$c = \frac{b}{s} \times d[s] = c \times d \quad [b]$$



se noi vogliamo sfruttare appieno una connessione, dobbiamo cercare di riempirla

Links:

è un qualsiasi collegamento tra due nodi, può essere un cavo di rame, una fibra ottica, un collegamento radio, piccione viaggiatore, ecc.

Un aspetto fondamentale dei link sono le **frequenze** (misurate in Hz - numero di oscillazioni al secondo):

il **periodo** è il tempo che impiega un'onda a compiere un'oscillazione completa.



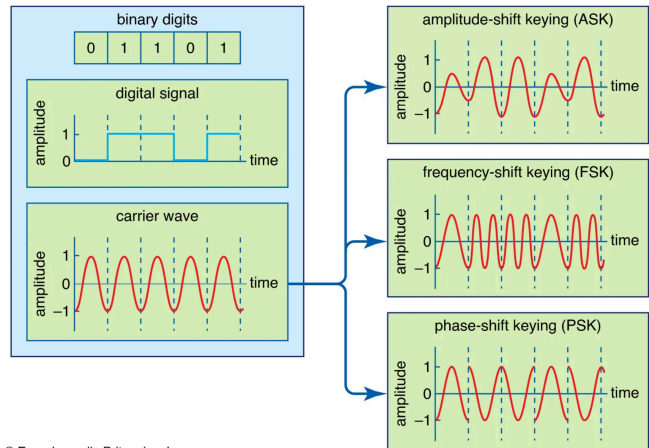
Encoding

Signals and Symbols

il segnale dello spettro elettromagnetico non è altro che una sinusoide, scomponibile in tre componenti:

- ampiezza (ASK: Amplitude)
- frequenza (FSK: Frequency)
- fase (PSK: Phase)

posso quindi codificare le informazioni variando questi tre componenti:



© Encyclopædia Britannica, Inc.

- se volessi lavorare sulla ampiezza, un'onda "alta" rappresenta un bit 1 e un'onda "bassa" rappresenta un bit 0.
- se volessi lavorare sulla frequenza, raddoppiando la frequenza potrei trasmettere un bit 1 e mantenendo la frequenza originale trasmettere un bit 0.
- se volessi lavorare sulla fase, ruotando la fase di 180 gradi potrei trasmettere un bit 1 e mantenendo la fase originale trasmettere un bit 0.

nulla vieta di eseguire più di una di queste operazioni contemporaneamente, permettendo di trasmettere più bit contemporaneamente.

posso anche usare una sinusoide composta da altre sinusoidi, permettendo di trasmettere più onde contemporaneamente, quindi più bit contemporaneamente.

chi trasmette sceglie uno **stato** (altezza del segnale, frequenza o fase) di un canale, chi riceve deve esserselo, questi stati sono chiamati **simboli**.

i simboli **portano** i bit, e possono essere qualsiasi cosa:

- Livelli di tensione (come nei circuiti)
- Ampiezza di una frequenza portante (AM)
- Spostamento di frequenza (FM)
- Spostamenti di fase (PSK. Molto utilizzato per la codifica digitale)
- Luce accesa/spenta. O diversi livelli di luce
- Lettere sul foglio. Ogni lettera + spazio è l'alfabeto
- Fonemi linguistici nella voce
- Sbuffi di fumo
- Macchie bianche/nere, come nei codici QR
- etc...



la velocità in cui si riescono a trasmettere i simboli è chiamata **baud rate** (simboli al secondo).

TEOREMA DI NYQUIST-SHANNON:

su un canale di larghezza di banda B (Hz), il massimo numero di simboli che si possono trasmettere è $2B$ per essere considerati un canale di comunicazione affidabile.

frequenza di banda:

la differenza tra la frequenza più alta e quella più bassa che un canale può trasmettere.

ma se io dividessi il canale in più sottocanali (raddoppiando la frequenza di banda), teoricamente potrei trasmettere infiniti simboli (se continuassi a dividere il canale in sottocanali), tuttavia ciò non è possibile a causa del **rumore**.

il rumore può essere indotto da una svariata serie di fattori, come ad esempio:

- espansioni termiche dei materiali
- interferenze elettromagnetiche
- precisione dei encoders
- etc...

se il rumore è troppo elevato, i simboli trasmessi possono essere alterati e ricevuti/decodificati come un altro simbolo.

di conseguenza, più posso limitare il rumore, più posso trasmettere simboli, quindi più posso trasmettere bit.

la **potenza** di un segnale si rappresenta con:

$$P_X = \lim_{T \rightarrow \infty} \frac{1}{T} \int_{-T/2}^{T/2} |x(t)|^2 dt$$

la potenza di un rumore **bianco** (omogeneo) viene rappresentato con:

$$P_n = \int_{f_1}^{f_2} E df = (f_2 - f_1) \cdot E$$

Con **E** si intende la densità della potenza (misurata in W / Hz)

la quantità di rumore è spesso espressa in **dB** (decibel), che è una scala logaritmica in base 10.

- un valore di 0 dB significa che il segnale e il rumore hanno la stessa potenza
- un valore positivo significa che il segnale ha più potenza del rumore
- un valore negativo significa che il rumore ha più potenza del segnale



TEOREMA DI SHANNON-HARTLEY

$$C = B \log_2(1 + S/N) \quad [b/s]$$

conseguenza:

per ottenere una capacità di trasporto maggiore, posso:

- aumentare la larghezza di banda (B)
- aumentare il rapporto segnale/rumore (S/N)
- aumentare entrambi

ma come si esegue l'encoding di un segnale?

NRZ encoding

come funziona:

- il ricevitore tiene una media dei segnali che ha visto fino ad ora
- utilizza la media per distinguere tra 0 e 1

problemi:

- se abbiamo troppi bit di seguito uguali, la media viene spostata verso quel valore, rendendo valori successivi difficili da distinguere.

clock recovery:

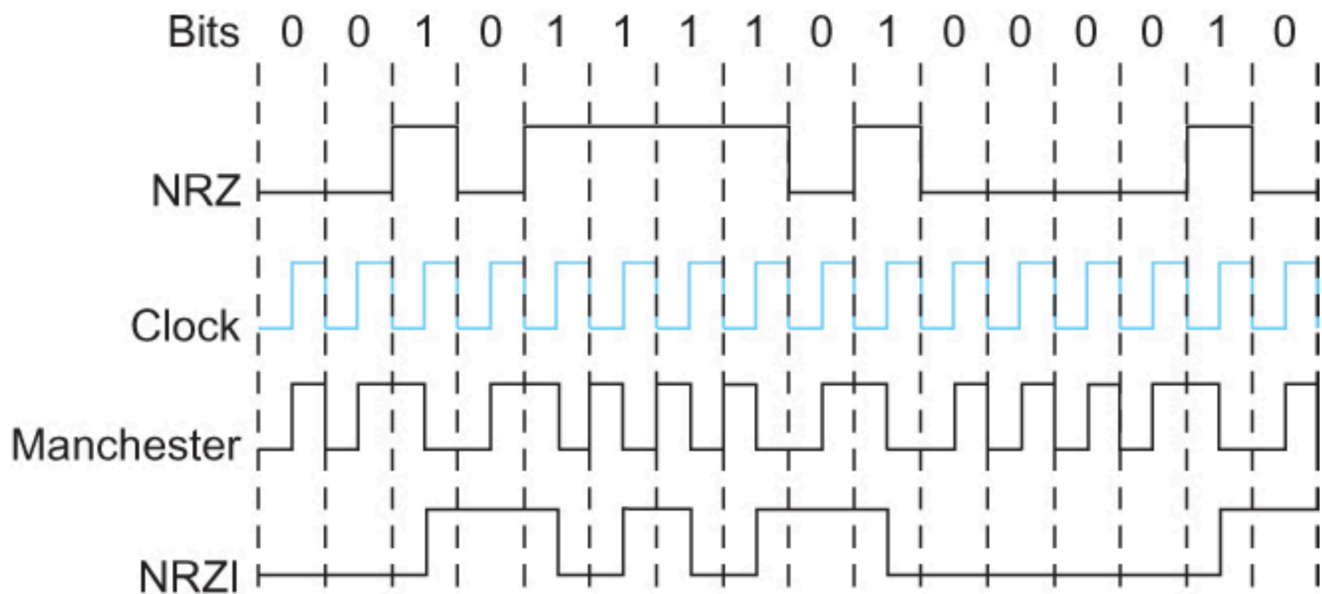
- è impossibile avere due clock perfettamente sincronizzati su lunghe distanze

NRZI encoding (Non Return to Zero Inverted)

- Sender makes a transition from the current signal to encode 1 and stay at the current signal to encode 0
- Solves for consecutive 1's, not for long consecutive 0's

Manchester encoding

- esegue una inversione del segnale a metà del bit, quindi se il bit è 1 il segnale passa da basso ad alto, se il bit è 0 il segnale passa da alto a basso
- questo garantisce di non perdere il conto (Si definisce auto-clocking)



saltato alcune parti perchè il pc si è scaricato

Framing

i dati vengono trasmessi strutturati (in pacchetti), dobbiamo quindi trovare il modo corretto di comporli.

Byte-oriented Protocols:

- BISYNC (Binary Synchronous Communication)
 - approccio Sentinel-based: abbiamo caratteri speciali che delimitano dei confini
- DDCMP (Digital Data Communication Protocol)
 - utilizza dei contatori di lunghezza

BISYNC

i dati sono delimitati da dei **caratteri speciali** (detti marcatori), quindi il ricevitore sa quando inizia e quando finisce un pacchetto.

Si usano solitamente gli ASCII control characters, come ad esempio:

- SYN (Synchronous Idle - 22)
- SOH (Start of Header - 1)
- STX (Start of Text - 2)
- ETX (End of Text - 3)
- DLE (Data Link Escape - 16)
- CRC (controllo errori)



Uno dei principali svantaggi è che la quantità di dati trasmessi **aumenta**, perchè i marcatori vengono trasmessi insieme ai dati.

nel caso peggiore la quantità di dati trasmessi può **raddoppiare**, perchè per ogni carattere inviato potrei dover inviare un carattere speciale (DLE) per indicare che il carattere successivo è un dato e non un marcatore.

In un contesto realistico, in uno scenario realistico, utilizzo il 70% della capacità del canale per trasmettere dati e il 30% per trasmettere marcatori.

DDCMP

ci sono i caratteri iniziali e finali del frame, seguiti da un **contatore di lunghezza**, che ci indica quanto è lungo complessivamente l'intero frame.

questo consente di trasmettere i dati con una quantità di overhead **minore** rispetto al BISYNC, tuttavia se il count è corrotto, il ricevitore non saprà più dove inizia e dove finisce il pacchetto.

Bit-oriented Protocols:

- HDLC (High-Level Data Link Control)
 - ha una sequenza di bit specifica (**01111110** - uno zero seguito da sei 1 e con uno 0 finale) che indica l'inizio e la fine del pacchetto, quindi il ricevitore sa quando inizia e quando finisce un pacchetto.

cosa succede se la sequenza che indica la lunghezza del pacchetto si trova anche all'interno dei dati?

nel HDLC se durante la trasmissione dei dati il ricevitore vede 5 bit consecutivi a 1, il ricevitore inserisce un bit a 0, quindi se il ricevitore vede 5 bit consecutivi pari a 1 e uno 0, sa che il bit successivo è un bit di **stuffing** e lo ignora, in questo modo si "rompono" sequenze che avrebbe indicato la fine del pacchetto (erroneamente)

PPP (Point-to-Point Protocol)

molto usato sulle linee seriali, deriva dal HDLC: utilizza sentinelle e bit-stuffing:

- usa **01111110** come Flag
- la dimensione del payload è variabile (default 1500 byte)

Error Detection

è possibile che durante la trasmissione di dati (per una serie infinita di variabili) alcuni bit vengano alterati, quindi è necessario un meccanismo per rilevare questi errori e correggerli. il processo si divide principalmente in:



- **detection**: il ricevitore rileva che c'è stato un errore, ma non sa quale bit è stato alterato
 - **Drop**: elimina il pacchetto e non lo passa al livello superiore come se non fosse mai stato inviato (se il pacchetto è critico il livello superiore non ricevendo il pacchetto può decidere di richiederlo nuovamente)
 - **Notify**: viene notificato al mittente che c'è stato un errore e il pacchetto deve essere ritrasmesso
- **Foward Error Correction**: il ricevitore rileva che c'è stato un errore e lo corregge con appositi algoritmi, quindi non deve richiedere al mittente di ritrasmettere i dati (fino a un certo limite).

idea di base:

chi trasmette aggiunge delle informazioni rindondanti (detti codici di controllo) e sono calcolati in funzione al messaggio

formati standard:

- CRC (Cyclic Redundancy Check)
 - standard più usato
 - usato in DDCMP, HDLC, CSMA/CD---
- Altri approcci:
 - checksum
 - parity bit
 - Two-Dimensional Parity

il metodo più semplice in assoluto è il **bit di parità**: ogni 7 bit di dati si aggiunge un bit di controllo, che aggiunto al numero di 1 presenti nei 7 bit di dati deve essere pari (parità pari) o dispari (parità dispari), se questo non avviene vuol dire che c'è stato un errore.

è estremamente semplice da implementare, ma non è molto affidabile, perchè se due bit (o qualsiasi numero pari di bit) vengono alterati, il bit di parità non rileverà l'errore.

una evoluzione di questo metodo è la **parità bidimensionale**, dove i bit di parità vengono calcolati sia sulle righe che sulle colonne, quindi se un bit viene alterato, il ricevitore può capire quale bit è stato alterato e correggerlo.

questo metodo consente anche di correggere alcuni errori, ed è in grado di rilevare errori multipli, ma non è infallibile, perchè se due bit vengono alterati nella stessa riga e nella stessa colonna, il ricevitore non sarà in grado di rilevare l'errore.

